# Drop, Reflect and Reuse a Quantum Temporary

ANONYMOUS AUTHOR(S)

We characterize safe drops by contextual equivalence. Without measurement, a safe drop means the temporary ends in the same pure state at every input, so at most one reflection restores it. The test extends to measurement records and loans and drives an OpenQASM 3 reuse compiler.

## 1 Specifying a safe drop

When a temporary qubit is no longer needed, we want to reuse it without disturbing the rest of the computation. The standard technique, uncomputation, runs the computation that produced a temporary backwards, which can take as many gates as the computation itself [14, §1]. Whether a temporary may be safely discarded is a separate question, which languages and tools such as Silq and Unqomp settle with conservative rules for automatic reclamation that can reject safe temporaries [2, §4.4], [14, §6.2].

Consider $p$ below, which allocates its only temporary `t`, of dimension $d$, in pure state $\psi$, runs a measurement-free computation $M$ on `(t,v)`, and drops `t`. We compare $p$ just before the drop with an independent witness $\mathcal{S}(P,U)$, a comparison program that prepares `t` using $P$ without the input `v`, while $U$ acts on `v`:

$$p(\mathtt{v}) = \texttt{new t : [d] @ ψ; M; drop t; return v}$$

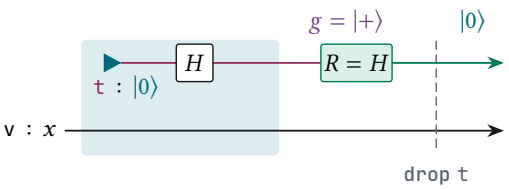
$$\mathcal{S}(P,U)(\mathtt{v}) = \texttt{new t : [d] @ ψ; P t; U v; drop t; return v}$$

We call the drop *safe* if $p \equiv \mathcal{S}(P,U)$ for one witness on all inputs, meaning that no context can distinguish the programs stopped before their drops, with `t` and `v` returned.

We show that at a safe drop, `t` ends in one fixed pure state $g$ at every input. From $g$ we construct a return gate $R$ that restores $\psi$ on `t` alone with at most one reflection. Triangles mark allocation and ground symbols end scope. The two examples below take a qubit `v`, with inserted return gates shown in green. The shaded box is the *exposed prefix* before the chosen `drop` *site*. A wire label `v` : $x$ gives the register and its input state. The exposed prefix's linear map $K$ sends $x$ to the joint state there, with `t` first.

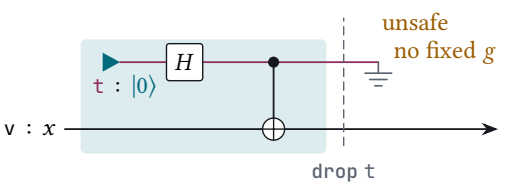
```
prog a (v : [2]) : [2] {
  new t : [2] @ |0>; H t;
  drop t; return v }
```

$K^{(a)}x = |+\rangle \otimes x$




```
prog b (v : [2]) : [2] {
  new t : [2] @ |0>; H t; CX t v;
  drop t; return v }
```

$K^{(b)}x = (|0\rangle \otimes x + |1\rangle \otimes Xx)/\sqrt{2}$




In our language, `drop` ends `t`'s scope without changing the joint state. Program (a) leaves `t` in $|+\rangle$ at every input, so $H$ alone restores `t` to $|0\rangle$. In program (b), `CX t v` flips `v` when `t` is $|1\rangle$, giving the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$ when `v` starts in $|0\rangle$. Both temporaries have the same lexical scope, so checking scopes would accept both drops. Silq and Unqomp's

`qfree` rules exclude $H$ because it takes basis states to superpositions, and reject both drops. Unqomp even rejects `H t; H t`, where `t` ends in $|0\rangle$ [14, §6.2].

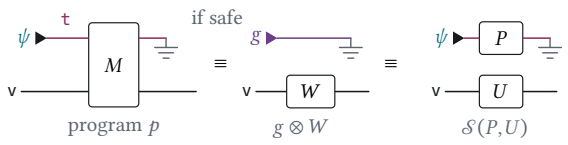
## 2 Checking the equivalence

$$K^{(a)} = \frac{1}{\sqrt{2}} \overbrace{\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}}^{H \otimes I} \cdot \overbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}}^{|0\rangle \otimes I} = \overbrace{\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}^{K^{(a)}} \begin{matrix} K_0 \\ K_1 \end{matrix} \quad \text{slices } I/\sqrt{2},\ I/\sqrt{2}$$

$$\overbrace{\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}}^{K^{(b)} = \mathrm{CX}\, K^{(a)}} \begin{matrix} K_0 \\ K_1 \end{matrix} \quad \text{slices } I/\sqrt{2},\ X/\sqrt{2}$$

To check the equivalence, we form the exposed prefix's matrix. For program (a), allocation $|0\rangle \otimes I$ acts before $H \otimes I$. We group rows by `t`'s basis value, so the blue *slice* $K_0$ gives $|0\rangle \otimes K_0 x$ and the gold $K_1$ gives $|1\rangle \otimes K_1 x$. Both terms remain in the joint state, so neither slice is selected.

A *context* is a closed program whose hole takes `v` to `(t,v)`. It supplies an input, possibly entangled with a reference, applies gates and reads all registers, recording outcome weights, as in




```
new v : [2] @ |0>; (t,v) = { □ }; CX t v; H t; return (t,v).
```

This context undoes program (b)'s gates and reads 00 with weight 1. An independent witness leaves `t` and `v` uncorrelated, so the same context reads 00 at most half the time. The witness prepares `t` in a fixed pure state $g$ without using the input. All input dependence lies in one map $W$ on the rest, so the witness's slice at basis value $i$ is $g_i W$, where $g_i$ is the coefficient of $|i\rangle$ in $g$. For $p$ to be equivalent to the witness, no context can distinguish their joint states before the drop, on any input. This requires the same slice form $K_i = g_i W$ for $p$, giving $Kx = g \otimes Wx$. In program (a), equal slices give $g = |+\rangle$ and $W = I$, while in program (b) no scalar turns $I$ into $X$.

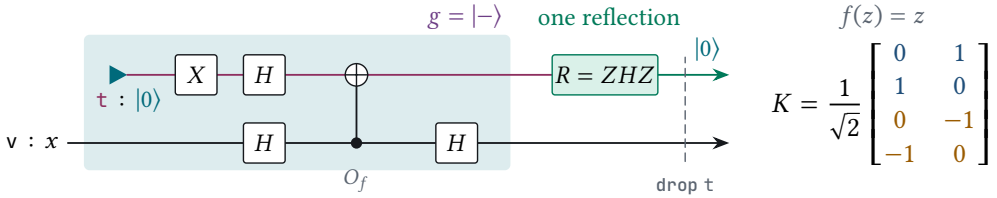
The witness prepares the dropped registers together as one *block*, without the input, and allocates matching registers for $p$'s other temporaries so contexts can compare the full joint state, including hidden registers.

Theorem 2.1 (without measurement). *Let $K$ be the exposed map before dropping allocated temporaries with pure allocation state $\psi$ in a well-typed, measurement-free program $p$. These are equivalent.*

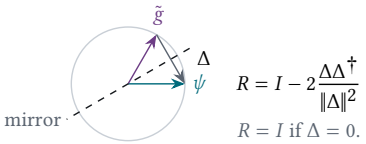
*(1) $p \equiv \mathcal{S}(P,U)$ for some independent witness.*
*(2) $K = g \otimes W$ for a unit vector $g$ on the block and an isometry $W$, a length-preserving map from the input to all other registers.*
*(3) One unitary on the block alone restores $\psi$ at every input.*

Deutsch–Jozsa shows why slices need only be proportional. Its oracle $O_f$ flips `t` when $f(z) = 1$. In the minus state $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, a flip only negates `t`, so the oracle leaves

it in $|-\rangle$ and gives the data state $|z\rangle$ a sign $(-1)^{f(z)}$. These signs define $D_f|z\rangle = (-1)^{f(z)}|z\rangle$. For one data qubit, the Hadamards give $W = HD_fH$, pictured for $f(z) = z$. The slices are $W/\sqrt{2}$ and $-W/\sqrt{2}$, proportional for every $f$, with $g = |-\rangle$. Reversing the preparation takes $H$ then $X$ on `t`. The reflection $R = ZHZ$ restores `t` in one gate, and both methods leave the data unchanged.




To build $R$, choose a phase representative $\tilde{g}$ of our *certificate* $g$ with real, nonnegative overlap with $\psi$, moving the phase into $W$. The difference $\Delta = \psi - \tilde{g}$ points from $\tilde{g}$ to $\psi$. If they coincide, no gate is needed. Otherwise the Householder reflection through the mirror orthogonal to $\Delta$ sends $\tilde{g}$ to $\psi$ [8, p. 340, Eqs. (3)–(4)]. For a general block, the compiler aligns the direction of $\Delta$ with $|0\cdots0\rangle$, flips that vector's sign, then undoes the alignment. This may take several QASM gates, but the circuit depends on the block's certificate rather than the producer's length. If $\Delta/\|\Delta\|$ is the uniform superposition, $-R$ is Grover's diffusion.



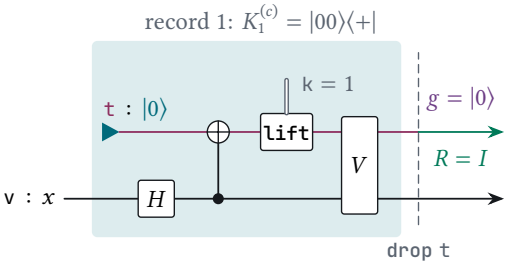

## 3 One measurement record at a time

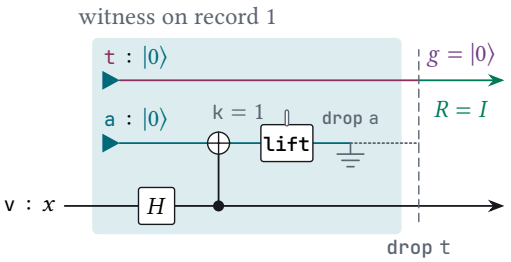
With measurement, we check each outcome sequence, or *record* $o$, separately. Our `lift[S] t` keeps `t` live and binds 1 for membership in a basis set $S$, or 0 otherwise. Only classical `if` guards read this outcome, as in dynamic lifting [6]. Thus `lift[{0}] t` returns 1 when `t` is read as 0. The exposed map $K_o$ stays unnormalized, so $\|K_o x\|^2$ is the record's probability at a normalized input $x$.

Program (c) uses a two-qubit unitary $V$ that fixes $|00\rangle$ and sends $|11\rangle$ to $(|01\rangle + |10\rangle)/\sqrt{2}$. Its independent witness measures a fresh helper qubit `a` instead of `t`:

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>; H v; CX v t;
  k = lift[{0}] t; V t v;
  drop t; return v }
```

```
prog witness (v : [2]) : [2] {
  new t : [2] @ |0>; new a : [2] @ |0>;
  H v; CX v a; k = lift[{0}] a;
  drop a; drop t; return v }
```



At input $x = \alpha|0\rangle + \beta|1\rangle$, measurement followed by $V$ leaves these unnormalized states:

- Record 1 (safe) leaves $(\alpha + \beta)|00\rangle/\sqrt{2}$, with weight $|\alpha + \beta|^2/2$. The temporary remains in $|0\rangle$, so $R = I$.
- Record 0 (unsafe) leaves $(\alpha - \beta)(|01\rangle + |10\rangle)/2$, with weight $|\alpha - \beta|^2/2$. Its nonzero outputs are Bell pairs, so this record has no pure certificate $g$.

On record 1, $K_1^{(c)} = |0\rangle\otimes W_1$ with $W_1 = |0\rangle\langle+|$. The map $W_1$ keeps only the $|+\rangle$ component of the input, so it never lengthens a vector and is a *contraction*. For example, $W_1$ gives $|0\rangle$ with weight 1 at $x = |+\rangle$, but $|0\rangle/\sqrt{2}$ with weight $1/2$ at $x = |0\rangle$. Normalizing would give $|0\rangle$ in both cases, hiding the dependence of record 1's weight on the input.

If we allocate the witness's `t` and `a` in $|0\rangle$ and apply `H v; CX v a`, these steps preserve total weight, so reproducing record 1 still requires measurement. They leave `t` in $|0\rangle$, while `v` and `a` have the same value in each basis term. Measuring the independent `t` would give the same probability at every input, so the witness measures `a`. Outcome 1 leaves `t` and `a` in $|0\rangle$ and `v` in $W_1 x$, with record 1's weight. Program (c)'s unsafe record 0 still prevents reuse.

The same construction works whenever $K_o x = g \otimes W_o x$ with $W_o$ a contraction. The helper's other outcome has the missing weight $\|x\|^2 - \|W_o x\|^2$. With fresh registers to match the input and output spaces, the two outcomes preserve norm and extend to a unitary $U$ [17, Thm. 2]. One `lift` selects $W_o x$, while $P$ prepares $g$ from $\psi$ on the block.

We call record $o$ *safe* and write $p \equiv_o \mathcal{S}(P,U)$ if no context distinguishes $p$'s record $o$ from the witness's kept outcome 1 before the drop, omitting only its fixed $|0\rangle$ helper factor. Both maps remain unnormalized, whatever the other records do.

Theorem 3.1 (on a selected record). *Let $K_o$ be the exposed map before dropping allocated temporaries on a record $o$ of a well-typed program $p$. These are equivalent.*
*(1) $p \equiv_o \mathcal{S}(P,U)$ for some independent witness using one* `lift`.
*(2) $K_o = g \otimes W_o$ for a unit vector $g$ on the block and a contraction $W_o$ to all other registers.*
*(3) One unitary on the block alone restores its pure allocation state $\psi$ at every input on $o$.*
*Zero maps satisfy the three conditions, and return gates may differ between records.*

Since reuse must preserve observable behavior, we might hope to decide safety from the *summed channel*, the mixture of returned-register and reference states over records [20, §2.3.2]. With `t` prepared in $|+\rangle$, measuring `t` in the standard basis after a controlled $Z$ on `(t,v)` is safe on both records, but measuring it before `v` controls a flip of `t` is unsafe on both. Both drop `t` and return `v`, inducing the same unread standard-basis measurement with weight $1/2$ for each record at every input.

## 4 Blocks and loans

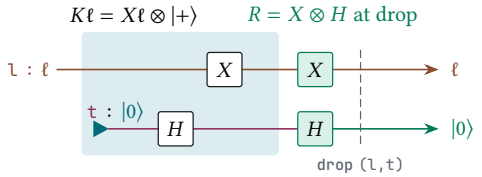
An allocated block in a fixed pure state is safe to drop even when its parts are mixed, as with a fresh Bell pair. We also consider blocks containing *loans*, registers borrowed in unknown states, possibly entangled with their caller, who expects them back unchanged.

The witness $\mathcal{S}(P,U)$ receives the unknown loan state $\ell$ as input and allocates temporaries in $\psi$, giving $J\ell = \ell \otimes \psi$. It applies $P$ to the block and lets $U$ act outside it. The fixed state $g$ becomes an isometry $B$ from loan to block. Safety now means $K = B \otimes W$, or

$K(\ell \otimes x) = B\ell \otimes Wx$, with loans first and $x$ the other inputs. A block unitary $P$ carries $J$ onto $B$, so $PJ = B$ gives the witness and $R = P^\dagger$ gives $RB = J$.

On record $o$, $K_o = B_o \otimes W_o$ and $R$ may depend on $o$. Without measurement, $R$ restores the loan's normalized joint state with its caller reference. This also holds on each occurring record when loan and reference are uncorrelated with other inputs. Otherwise measuring correlated other inputs can change that state. The diagram below gives $K$ and $R$ for `lend`, whose unit return is `() : 1`.

```
prog lend (loan l : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X l;
  drop (l,t); return () }
```




## 5 Scope and further work

We prove progress and preservation for a linear, first-order abstract machine with lifting and classical guards. For reuse, it restores safe blocks and pools temporaries for `new` of matching dimension and state, preserving records, weights and the reduced state of returned registers on each record. Our exact Haskell checker tests each record's basis inputs for a common temporary state or loan isometry $B$. Checking is exponential in the number of qubits. With basis allocations and $X$, `CX` and Toffoli, safety of drops without loans is coNP-complete, even without measurement and with one drop (compare [11, Cor. 3.1]).

Our OpenQASM 3 compiler runs this check, computes numerical return gates, and reuses blocks safe on every record. Measuring and resetting a qubit to $|0\rangle$ also permits reuse [4, 9], but destroys an unknown loan state. Only basis allocations compile, program (c)'s $V$ is unsupported, and no loan may be dropped after a `lift` or an `if`. For general loan returns, the compiler checks $RB = J$ entrywise within $10^{-9}$.

Qurts and Reqomp reclaim by uncomputing [7, 15], while our test constructs a return from the state at the drop. It certifies the balanced Deutsch–Jozsa temporary that Qutes does not reclaim [5, Defs. 2–6]. Twist verifies annotated purity through static analysis and runtime separability tests [22, §1.3], but constructing one return also requires the state to be independent of the input. The gate we construct provides the restoration that ReQWIRE verifies [16, §4] and that Q# and fractional types require [3, 13].

For loans, local rotations repair separable errors on a dirty ancilla [10, §5.2, Alg. 2], whereas our return undoes an isometry $B$ of the whole block. QBorrow requires the loan unchanged after each denoted operation [18, Def. 5.1] and Bona after each borrowing body [21, Def. 3.2], a handback property that separation logics verify deductively [12, 19].

Our language excludes loops and quantum recursion, so safety after them remains open. We also allow all unitaries as gates, although a fixed set may lack the return. Circuits over $X$, `CX`, Toffoli and $H \otimes H$ have dyadic entries [1, Cor. 5.6]. Applying $H \otimes H$ to `(t,v)` with `t` allocated in $|0\rangle$ gives a safe drop, yet restoring `t` alone requires non-dyadic entries $\pm 1/\sqrt{2}$.

## References


[1] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-Theoretic Characterizations of Some Restricted Clifford+T Circuits. *Quantum* 4, Article 252 (2020). doi:10.22331/q-2020-04-06-252

[2] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '20)*. ACM, 286–300. doi:10.1145/3385412.3386007

[3] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[4] Matthew DeCross, Eli Chertkov, Megan Kohagen, and Michael Foss-Feig. 2023. Qubit-Reuse Compilation with Mid-Circuit Measurement and Reset. *Physical Review X* 13, 4 (2023), 041057. doi:10.1103/PhysRevX.13.041057

[5] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538v1 [quant-ph]

[6] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 11 (2023), 26 pages. doi:10.1145/3571204

[7] Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 6 (2025), 28 pages. doi:10.1145/3704842

[8] Alston S. Householder. 1958. Unitary Triangularization of a Nonsymmetric Matrix. *J. ACM* 5, 4 (Oct. 1958), 339–342. doi:10.1145/320941.320947

[9] Fei Hua, Yuwei Jin, Yanhao Chen, Suhas Vittal, Kevin Krsulich, Lev S. Bishop, John Lapeyre, Ali Javadi-Abhari, and Eddy Z. Zhang. 2023. CaQR: A Compiler-Assisted Approach for Qubit Reuse through Dynamic Circuit. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '23)*. 59–71. doi:10.1145/3582016.3582030

[10] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification – CAV 2026, Part III (Lecture Notes in Computer Science, Vol. 16684)*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16

[11] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. *Proceedings of the ACM on Programming Languages* 10, OOPSLA2 (2026), 2270–2298. doi:10.1145/3839528

[12] Yusuke Matsushita, Kengo Hirata, Ryo Wakizaka, and Emanuele D'Osualdo. 2026. RapunSL: Untangling Quantum Computing with Separation, Linear Combination and Mixing. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 6 (2026). doi:10.1145/3776648

[13] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[14] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[15] Anouk Paradis, Benjamin Bichsel, and Martin Vechev. 2024. Reqomp: Space-Constrained Uncomputation for Quantum Circuits. *Quantum* 8 (2024), 1258. doi:10.22331/q-2024-02-19-1258

[16] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[17] P. L. Robinson. 2018. Julia Operators and Halmos Dilations. arXiv:1803.09329. doi:10.48550/arXiv.1803.09329

[18] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. Borrowing Dirty Qubits in Quantum Programs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. 274–289. doi:10.1145/3779212.3790134

[19] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. An Interpretation of Bunched Logic for Reasoning about Heap-Manipulating Quantum Programs. arXiv:2409.10153v2 [cs.PL]

[20] John Watrous. 2018. *The Theory of Quantum Information.* Cambridge University Press. doi:10.1017/9781316848142

[21] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765v1 [cs.PL]

[22] Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: Sound Reasoning for Purity and Entanglement in Quantum Programs. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 30 (2022), 32 pages. doi:10.1145/3498691